Hito 0 - Automatas finitos: enteros positivos

Jonathan Javier Pietracci

22 de julio de 2026

Creando mi compilador en C;

1. Lenguaje y alfabeto

Sea el alfabeto de digitos decimales

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

El lenguaje de los enteros positivos en notación decimal es el conjunto de cadenas sobre Σ que cumplen:

1. el primer simbolo es un digito del 1 al 9
2. los simbolos siguientes pueden ser cualquiera de los 10 digitos.

2. Gramática formal

$$\begin{aligned}
\text{Numero} &\to \text{PrimerDigito Resto} \\
\text{Resto} &\to \varepsilon \mid \text{SiguienteDigito Resto} \\
\text{PrimerDigito} &\to 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \\
\text{SiguienteDigito} &\to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}$$

donde ε denota la cadena vacia.

3. Autómata finito determinista (AFD)

Definición 3.1. Un automata finito determinista es una quíntupla $M = (S, I, f, s_0, F)$ donde S es un conjunto finito de estados, I es el alfabeto de entrada, $f : S \times I \to S$ es la función de transición, $s_0 \in S$ es el estado inicial y $F \subseteq S$ es el conjunto de estados de aceptación.

Para nuestro lenguaje:

$$S = \{q_0, q_1, q_2\} \quad I = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, \quad s_0 = q_0, \quad F = \{q_1\}$$

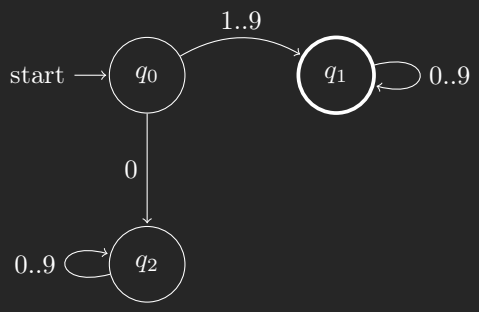
La función de transición f es:

$$
\begin{aligned}
&f(q_0, d) = q_1 \quad \text{si } d \in \{1, \ldots, 9\},\\
&f(q_0, 0) = q_2,\\
&f(q_1, d) = q_1 \quad \text{para todo } d \in I,\\
&f(q_2, d) = q_2 \quad \text{para todo } d \in I \quad \text{(estado de rechazo hundido).}
\end{aligned}
$$

Estado	0	1	2	3	4	5	6	7	8	9
q_0 (inicial)	q_2	q_1	q_1	q_1	q_1	q_1	q_1	q_1	q_1	q_1
q_1 (aceptación)	q_1	q_1	q_1	q_1	q_1	q_1	q_1	q_1	q_1	q_1
q_2 (rechaza)	q_2	q_2	q_2	q_2	q_2	q_2	q_2	q_2	q_2	q_2

Tabla 1: Función de transición f. Doble círculo = estado de aceptación.

3.1. Diagrama de estados



3.2. Verificación por simulación

Recorremos la cadena símbolo a símbolo usando la transición extendida.

Cadena	Recorrido de estados	Veredicto
`7`	$q_0 \to 7q_1$	acepta
`42`	$q_0 \to 4q_1 \to 2q_1$	acepta
`40`	$q_0 \to 4q_1 \to 0q_1$	acepta
`102`	$q_0 \to 1q_1 \to 0q_1 \to 2q_1$	acepta
`0`	$q_0 \to 0q_2$	rechaza
`007`	$q_0 \to 0q_2 \to 0q_2 \to 7q_2$	rechaza

Tabla 2: El AFD reconoce exactamente los enteros positivos sin ceros a la izquierda.

4. Gramáticas formales y jerarquía de Chomsky

Las gramáticas se clasifican en cuatro tipos según la forma de sus reglas:

1. tipo 0: sin restricciones
2. tipo 1: sensibles al contexto (A entre símbolos vecinos).
3. tipo 2: libres de contexto (un solo símbolo a la izquierda).
4. tipo 3: regulares (reconocibles por un autómata finito).

5. Teoría de lenguajes formales

Un lenguaje formal L sobre un alfabeto Σ es un conjunto de cadenas de Σ. La cerradura de Kleene Σ^* es el conjunto de todas las cadenas sobre Σ, incluyendo la cadena vacía.

Nuestro lenguaje de enteros positivos es un subconjunto propio de Σ^*:

$$L = \{w \in \Sigma^* \mid \text{el primer símbolo de } w \text{ es } 1, \ldots, 9\}.$$

No coincide con Σ^* porque Σ^* contiene cadenas como `0`, `00`, `007`, que nuestro AFD rechaza. Como L es reconocido por un autómata finito, es un lenguaje regular.

6. Inducción estructural

Demostramos por inducción sobre la construcción de las cadenas que todo elemento del lenguaje generado por nuestra gramática empieza con un dígito del 1 al 9

Demostración. **Caso base.** La cadena más corta se obtiene con Resto $\to \varepsilon$, lo que da un PrimerDigito, por ejemplo 1. Esta cadena empieza con 1, luego cumple la propiedad.

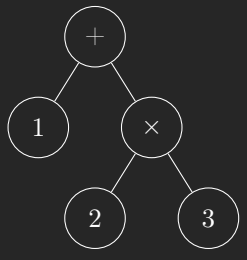
Paso inductivo. Supongamos que una cadena w cumple la propiedad (empieza con 1 al 9). Al aplicar Resto $\to$ SiguienteDigito Resto se agrega un dígito $d \in \{0, \ldots, 9\}$ al final de w. Como d se agrega al final, el primer símbolo de w no cambia; por tanto wd también empieza con 1 al 9.

Por inducción, toda cadena generada por la gramática cumple la propiedad. □

7. Árboles y el árbol de sintaxis (AST)

Un árbol es un grafo conexo y acíclico con un nodo raíz; cada nodo tiene un único padre, salvo la raíz. Los nodos sin hijos son las hojas.

En el compilador, una expresión se representa como un árbol de sintaxis abstracta (AST). La expresión $1 + 2 \times 3$ se esctructura así:



Las hojas son $1, 2, 3$. Al evaluar de las hojas hacia la raíz se calcula primero $2 \times 3 = 6$ y luego $1 + 6 = 7$, respetando la precedencia.

8. Tablas hash y la tabla de símbolos

8.1. Función de hash

Una función de hash es una aplicación

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}$$

que asigna a cada clave del universo U (por ejemplo, todas las cadenas posibles) un índice en un arreglo de m casillas. La función h se *define*; no es una construcción condicional: a cada clave le corresponde un único índice por la fórmula elegida.

8.2. Principio del palomar

Si $|U|$ (la cantidad de claves posibles) es infinito y m es finito, entonces forzosamente existen al menos dos claves distintas con el mismo índice. Esto no es un error de implementación: es consecuencia del principio del palomar (si se ponen más palomares que casillas alguna casilla recibe al menos dos). Por tanto, las *colisiones* son inevitables y deben *resolverse*, no evitarse.

8.3. El módulo como herramienta matemática

Para claves enteras se usa

$$h(k) = k \bmod m,$$

donde $k \bmod m$ es el resto único de dividir k entre m, y por definición cumple $0 \leq (k \bmod m) < m$. Así el índice siempre cae en el rango del arreglo. En la práctica se elige m primo para que la distribución no descarte bits (si m fuera potencia de 2, $k \bmod m$ usaría solo los últimos bits y agruparía claves con patrones similares).

8.4. Claves que no son enteras

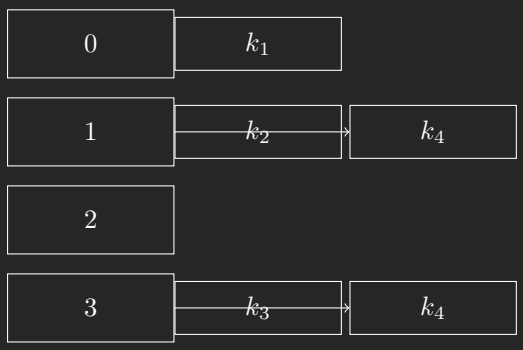
Si la clave es un *string* (como los identificadores del compilador), el módulo no aplica directamente porque opera sobre enteros. Hay que convertir primero el string en un entero que dependa de *todos* sus caracteres y de su orden:

$$e = 0; \quad \text{para cada caracter } c: \; e = e \cdot 256 + \text{código}(c)$$

Por ejemplo, `'ab'` (códigos 97 y 98) da $e = 97 \cdot 256 + 98 = 24930$. Entonces $h(\texttt{'ab'}) = 24930 \bmod m$. La misma cadena produce siempre el mismo entero, y por tanto la misma casilla.

8.5. Resolución de colisiones

- **Encadenamiento:** cada casilla contiene una lista enlazada. Si una segunda clave cae en una casilla ocupada, se agrega a la lista. Pueden guardarse más de m claves (el límite es la memoria).
- **Direccionamiento abierto:** cada casilla guarda una sola clave. Si la casilla i está ocupada, se prueba $i+1$, $i+2$, ... hasta hallar una libre. A lo sumo caben m claves; la tabla se satura cuando las casillas están llenas.



El dibujo muestra encadenamiento: las casillas 1 y 3 tienen listas enlazadas (k_5 cuelga de k_2; k_4 cuelga de k_3).

8.6. Conexión con el compilador

La tabla hash será la *tabla de símbolos*: mapea identificadores reconocidos por el lexer (Hito 1) a su información. El lexer produce el string, se aplica la conversión a entero y el módulo, y se resuelve la colisión con la estrategia elegida.

9. Binario y Sistema Posicional

Antes de hablar de "binario", necesitamos entender `qué es un sistema posicional`. Es una idea poderosa: representar cantidades usando `posiciones` y `pesos`.

la definición formal dice:

Definición 9.1. Un entero en base b se representa como

$$N = d_{k-1} \cdot b^{k-1} + d_{k-2} \cdot b^{k-2} + \cdots + d_1 \cdot b^1 + d_0 \cdot b^0$$

donde $0 <= d_i < b$ para cada i.

Ejemplo en base 10: el número 347_{10} se descompone como:

Posición	2	1	0
Dígito d_i	3	4	7
Peso b^i	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$

$$347_{10} = 3 \cdot 100 + 4 \cdot 10 + 7 \cdot 1 = 347$$

la base b define: (1) cuántos dígitos posibles hay (del 0 al b-1) y (2) cuánto 'vale' cada posición (potencias de b).

9.1. Binario: base 2

En base 2 solo existen dos dígitos: $\{0, 1\}$. Los pesos son potencias de 2:

Posición	5	4	3	2	1	0
Peso 2^i	32	16	8	4	2	1

Ejemplo: representar 21_{10} en binario. Se descompone $21 = 16 + 4 + 1$:

Posición	4	3	2	1	0
Peso 2^i	16	8	4	2	1
Dígito	1	0	1	0	1

$$21_{10} = 10101_2$$

9.2. Conversión decimal a binario

Definición 9.2. Para convertir un entero positivo n a binario, se aplican divisiones sucesivas por 2. Los restos, leídos de último a primero, forman los dígitos binarios (del LSB al MSB).

Ejemplo: 47_{10} a binario.

División	Cociente	Resto
$47 \div 2$	23	1
$23 \div 2$	11	1
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$2 \div 2$	0	1

Los restos de abajo hacia arriba: $47_{10} = 101111_2$.

9.3. Conversión binario a decimal

Definición 9.3. Para convertir $d_{k-1} \cdots d_1 d_0$ a decimal, se calcula $\sum_{i=0}^{k-1} d_i \cdot 2^i$.

Ejemplo: 110011_2 a decimal.

Posición	5	4	3	2	1	0
Dígito	1	1	0	0	1	1
Peso 2^i	32	16	8	4	2	1

$$32 + 16 + 0 + 0 + 2 + 1 = 51_{10}$$

9.4. Hexadecimal: base 16

Definición 9.4. El sistema hexadecimal usa 16 dígitos: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$. Cada dígito hex equivale a exactamente 4 bits.

Hex	Binario	Decimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

Binario a hex: separar en nibbles (grupos de 4 bits) y reemplazar cada grupo.
$11010101_2 = \underbrace{1101}_{D}\ \underbrace{0101}_{5} = D5_{16}$
Hex a binario: cada dígito hex ->4 bits.
$3F_{16} = \underbrace{3}_{0011}\ \underbrace{F}_{1111} = 00111111_2$

9.5. Notación de bases

Para desambiguar la base de un número se usa subíndice: 101_2 (cinco en binario), 101_{10} (ciento uno en decimal).

En C++ se usa prefijo: `0b101` (binario), `0xD5` (Hexadecimal), `101` (decimal).

10. Aritmética binaria

10.1. Suma binaria

La suma binaria sigue las mismas reglas que la suma decimal, con la salvedad de que 1 + 1 = 10 (cero con acarreo de 1).

A	B	Resultado	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Se suma de derecha a izquierda (LSB → MSB), propagando el carry. Si al final queda un carry, se escribe como bit adicional.

Ejemplo: $1101_2 + 1011_2$

	1	1	1	
+	1	0	1	1
1	1	0	0	0

$13 + 11 = 24011000_2$ ✓

10.2. Complemento a 2

Definición 10.1. El complemento a 2 de un número binario de n bits se obtiene:

1. Invertir todos los bits (complemento a 1).
2. Sumar 1 al resultado.

$$-x = \sim x + 1$$

Con n bits, el rango de enteros con signo es $[-2^{n-1},\ 2^{n-1} - 1]$.

Ejemplo -5 con 4 bits.

1. $5 = 0101$
2. Invertir: 1010
3. Sumar 1: 1011

$-5_{10} = 1011_2$ ✓

10.3. Tabla completa de 4 bits

Decima	Binario	Decimal	Binario
0	0000	-1	1111
1	0001	-2	1110
2	0010	-3	1101
3	0011	-4	1100
4	0100	-5	1011
5	0101	-6	1010
6	0110	-7	1001
7	0111	-8	1000

10.4. Resta por suma

La resta se implementa como suma con el negativo:

$$A - B = A + (-B)$$

Elcarry que excede el ancho de bits se descarta.

Ejemplo: $6 - 4$ con 4 bits. $6 = 0110$, $-4 = 1100$ (invertir $0100 \rightarrow 1011$, sumar $1 \rightarrow 1100$).

$$0110 + 1100 = 10010 \rightarrow 0010 = 2_{10} \quad ✓$$

10.5. Overflow

Definición 10.2. Hay overflow signed cuando el carry de entrada al MSB es distinto del carry de salida del MSB. El resultado no cabe en el rango $[-2^{n-1},\ 2^{n-1}-1]$.

Ejemplo: $7 + 1 = 8$ con 4 bits (rango $[-8, 7]$).

$0111 + 0001 = 1000$. El MSB indica negativo (-8), pero la suma real es $+8$. Hay overflow.

10.6. Un solo cero

En complemento a 2, $-0 = 0000_2$ (mismo que $+0$). No existe el doble cero, a diferencia del complemento a 1.